ABC Trading ERP

System Architecture Design

Unified ERP · POS · Sales Rep (B2B) · E-commerce

Business Overview

ABC Trading Co., Ltd. — FMCG wholesale & retail operator across 3 sales channels

Channel	Model	Payment Timing
POS	Walk-in retail store	Immediate at counter
Sales Rep (B2B)	Field sales to wholesale customers	Credit terms (Net 30 / 60)
E-commerce	Online web / mobile	Before shipment

> ***Problem:*** *Three channels operate independently — inconsistent data, duplicated processes, no unified view of inventory or AR.*
> ***Solution:*** *One ERP system. Single source of truth.*

System Objectives

- 📦 **Centralize** — sales, inventory, and financials in a single database
- 🔄 **Standardize** — shared order, payment, and fulfillment workflows across channels
- 👁️ **Transparency** — real-time stock levels, order status, and AR balances
- 🏗️ **Extensibility** — add channels, branches, or markets without redesign

High-Level Architecture

```
flowchart TD
    POS["🖥️ POS Terminal"]          --> OML
    SR["📱 Sales Rep Mobile/Web"]  --> OML
    EC["🌐 E-commerce Web/App"]    --> OML

    OML["Order Management Layer — unified"]

    OML --> INV["📦 Inventory Module"]
    OML --> PRC["🏷️ Pricing & Catalog"]
    OML --> ACC["💰 Accounting — AR/AP"]

    INV --> DB[("🗄️ PostgreSQL\nShared Database")]
    PRC --> DB
    ACC --> DB

    style OML fill:#4299e1,color:#fff,font-weight:bold
    style DB  fill:#2d3748,color:#fff
```

> ***Modular monolith*** *— single deployable unit, cleanly separated domains. No distributed systems complexity until it's needed.*

Data Model — Core Entities

```
erDiagram
    CUSTOMERS    ||--o{ ORDERS       : "places"
    STORES       ||--o{ ORDERS       : "originates"
    EMPLOYEES    ||--o{ ORDERS       : "processed by"
    ORDERS       ||--o{ ORDER_ITEMS  : "contains"
    PRODUCTS     ||--o{ ORDER_ITEMS  : "included in"
    PRODUCTS     ||--o{ INVENTORY    : "tracked in"
    WAREHOUSES   ||--o{ INVENTORY    : "stores"
    STORES       ||--o{ WAREHOUSES   : "contains"
    ORDERS       ||--o{ PAYMENTS     : "paid via"
    ORDERS       ||--o| INVOICES     : "invoiced as"
    QUOTATIONS   ||--o| ORDERS       : "converts to"
    STORES       ||--o{ CASH_SHIFTS  : "has"
    EMPLOYEES    ||--o{ CASH_SHIFTS  : "operates"
```

> *18 entities total — simplified view shown. Full diagram in* `er-diagram.md`.

POS Workflow

```
sequenceDiagram
    participant CSH as Cashier
    participant ERP as ERP System
    participant INV as Inventory

    CSH->>ERP: Open shift (store_id + opening_cash)
    Note over ERP: CASH_SHIFTS record created

    CSH->>ERP: Scan products at counter
    ERP->>INV: Check stock availability
    INV-->>ERP: Stock confirmed + price

    CSH->>ERP: Tender payment (Cash / Card / QR)
    ERP->>ERP: Create ORDER (store_id + cashier_id)
    ERP->>INV: Deduct qty_on_hand
    ERP->>CSH: Issue receipt — Order: Completed

    CSH->>ERP: Close shift (closing_cash)
    Note over ERP: CASH_SHIFTS reconciled
```

B2B Workflow — Quotation to Sales Order

```
sequenceDiagram
    participant C   as Customer
    participant ERP as ERP System
    participant INV as Inventory
    participant ACC as Accounting

    C->>ERP: Submit RFQ
    ERP->>INV: Check availability + lead time
    ERP->>ERP: Create QUOTATION_ITEMS (negotiated price per product)
    ERP->>C: Send Quotation

    alt Customer Approves
        C->>ERP: Approve quotation
        ERP->>ERP: Convert → ORDER + ORDER_ITEMS (quotation_id set)
        ERP->>INV: Reserve stock (qty_reserved++)
        ERP->>ACC: Issue tax invoice (credit terms)
        ERP->>C: Sales Order + Invoice
        C->>ACC: Pay within credit period
    else Rejected / Expired
        ERP->>INV: Release reservation
        Note over ERP: Quotation → Cancelled
    end
```

Key Design Decisions

Decision	Rationale
INVENTORY as junction table	PRODUCTS ↔ WAREHOUSES many-to-many; stock never stored in PRODUCTS
`qty_available` not persisted	Computed as `qty_on_hand - qty_reserved` at query time — eliminates stale reads under concurrent writes
PAYMENT_TRANSACTIONS separate from PAYMENTS	PAYMENTS = intent; TRANSACTIONS = gateway attempts. Retries insert a new row — no mutation, full idempotency
QUOTATIONS independent of ORDERS	Eliminates circular FK. `ORDERS.quotation_id` (nullable) references the source — lifecycle is unambiguous
`store_id` + `cashier_id` nullable on ORDERS	Single unified ORDERS table — POS populates both fields; B2B and e-commerce leave them NULL
Pessimistic lock on INVENTORY	`SELECT FOR UPDATE` on reservation prevents oversell under concurrent POS and online orders

Scalability Strategy

```
flowchart LR
    A["🏗️ Phase 1\nModular Monolith\nPostgreSQL + indexes"]
    B["📖 Phase 2\nRead Replicas\n+ Redis Cache"]
    C["⚙️ Phase 3\nExtract Microservices\n(as bounded contexts grow)"]

    A -->|"read load grows"| B
    B -->|"domain autonomy needed"| C

    style A fill:#c6f6d5,color:#22543d
    style B fill:#bee3f8,color:#2a4365
    style C fill:#fefcbf,color:#744210
```

- **Phase 1** — Single deploy; domain modules enforce separation. Indexes on `orders(channel_id, status)`, `inventory(product_id, warehouse_id)`
- **Phase 2** — Read replicas for dashboards; Redis for inventory cache and POS session
- **Phase 3** — Extract Order, Inventory, Payment as services. Apply **CQRS** and **Event Sourcing** where audit depth matters

Conclusion

✅ What the design gets right
Unified order table across 3 channels — `channel_id` + nullable FKs
Inventory concurrency via pessimistic locking + DB-level constraints
Payment idempotency — `attempt_no` + gateway key, no mutation
Append-only audit tables — full traceability with zero update risk
POS operations cleanly added — `STORES`, `EMPLOYEES`, `CASH_SHIFTS`
Monolith-first — simple to deploy, clear path to microservices

> ***Design philosophy:*** *Start simple. Be explicit about trade-offs. Scale when the data proves it's needed.*